Strata — Kiến trúc hệ thống

Knowledge Base as Code

strata-kb v1.37.0 · 2026-11-14

1. Mục đích và phạm vi

Strata biến các tài liệu tham chiếu dài thành một cơ sở tri thức mà con người review được bằng pull request, còn AI agent truy vấn đúng đoạn cần dùng.

Tài liệu này mô tả **hệ thống được xây dựng như thế nào**: mô hình dữ liệu, các thành phần, đảm bảo mà mỗi thành phần cung cấp, và các mô hình triển khai. Đối tượng đọc là kiến trúc sư, platform engineer và technical lead đang đánh giá hoặc vận hành Strata. Hướng dẫn thao tác hằng ngày nằm ở bốn tài liệu hướng dẫn theo vai trò (hub, child, ba, dev) đi kèm.

Strata không gắn với lĩnh vực nào. Không có chỗ nào trong thiết kế mã hoá một ngành, một họ tài liệu hay một ngôn ngữ cụ thể. Đơn vị công việc là "một tài liệu dài mà chi tiết của nó quan trọng"; nội dung tài liệu nói về cái gì không bao giờ đi vào code.

1.1 Ràng buộc thiết kế

Bốn ràng buộc dưới đây định hình mọi quyết định trong tài liệu này.

Ràng buộc	Hệ quả trong thiết kế
Chi tiết phải sống sót qua bước tóm tắt	Bảng được code trích xuất, không bao giờ để model viết lại, và được diff tự động ở mỗi lần build
Review phải làm được mà không cần công cụ đặc biệt	Kho lưu là YAML và Markdown thuần trong Git; bề mặt review là pull request thông thường
Tra cứu phải đủ rẻ để chạy cho mọi câu hỏi	Bốn lớp, nên một truy vấn nạp một lát cắt chứ không nạp cả kho; không có lời gọi model nào trong lúc tra cứu
Tri thức phải được chia sẻ, không nhân bản	Hub giữ một bản duy nhất của mỗi tài liệu; các repo khác tham chiếu tới chứ không ingest lại

1.2 Strata không phải là gì

- **Không phải database.** Không có server giữ trạng thái. Mọi artefact là một file được quản lý phiên bản, và mọi truy vấn đọc bản làm việc của một cây Git.
- **Không phải pipeline RAG lấy vector store làm nguồn chân lý.** Chỉ mục tìm kiếm là dẫn xuất và có thể bỏ đi; Markdown mới là nguồn chân lý và dựng lại được chỉ mục bất cứ lúc nào.
- **Không phải hệ thống tài liệu tự vận hành.** Model soạn thảo phần văn xuôi. Luôn có người duyệt trước khi nội dung trở nên truy cập được.

2. Mô hình dữ liệu

2.1 Bốn lớp

Một kho Strata giữ mỗi tài liệu ở bốn mức chi tiết cùng lúc.

Lớp	Artefact	Nội dung	Do ai ghi
L0	`.kb/index.yaml`	Mỗi tài liệu một dòng: id, revision, tags, mô tả ≤ 30 từ	`kb ingest`, model tinh chỉnh
L1	`.kb/<doc>/_manifest.yaml`	Mỗi section một dòng: id, tiêu đề, tóm tắt ≤ 25 từ, status, số token, hash nội dung	`kb ingest` dựng khung, `kb summarize` điền
L2	`.kb/<doc>/<part>.md`	Văn xuôi cô đọng **bằng chính ngôn ngữ của tài liệu gốc**, mọi bảng giữ nguyên văn	`kb summarize` (văn xuôi) và code (bảng)
L3	`.kb/<doc>/<part>.raw.md`	Toàn văn trích xuất, không cắt gì	`kb ingest`, không bao giờ là model

Bốn lớp không phải là bốn lựa chọn thay thế nhau, mà là một đường đi xuống. Truy vấn duyệt L0 để chọn tài liệu, L1 để chọn section, rồi nạp L2 của vài section thực sự liên quan. L3 chỉ được mở khi người gọi cần nguyên bản không rút gọn.

```
 L0  index.yaml          ~200 token       "có những tài liệu nào"
  |
L1  _manifest.yaml      ~10^4 token      "có những section nào"
  |
L2  <part>.md           ~10^5 token      "section đó nói gì"
  |
L3  <part>.raw.md       ~10^5 token      "bản gốc nói chính xác điều gì"
```

Mô hình chi phí suy ra trực tiếp: một lượt tra cứu cần ba section thì trả giá cho L0, một lát cắt L1 đã lọc theo tag, và ba section L2 — không trả giá cho cả tài liệu.

2.2 Section

Section là đơn vị nguyên tử của trích dẫn và truy hồi. Mỗi section có:

- **Một id ổn định**, suy ra từ chính cách đánh số của tài liệu gốc (`5.129`, `ch2`, `appendix-3`). Id là khoá trích dẫn nên không bao giờ được gán lại; một section bị đánh số lại được mô hình hoá thành một section bị xoá và một section mới được thêm.
- **Một status**: `pending` (chưa có tóm tắt) → `summarized` (model đã viết) → `reviewed` (người đã duyệt, có ghi ai duyệt và duyệt lúc nào).

- **Hai hash nội dung**: `l3_sha256` cố định bản gốc mà bản tóm tắt được viết dựa trên, và `reviewed.l2_sha256` cố định bản tóm tắt mà người đã duyệt. Một trong hai thay đổi mà cái kia không đổi là lỗi build, không phải trôi dạt âm thầm.

2.3 Bất biến về bảng

Đây là đảm bảo chịu lực của toàn hệ thống.

> Văn xuôi có thể được model cô đọng. **Bảng thì không.** Mọi bảng đều do code trích từ nguồn, ghi y hệt nhau vào L2 và L3, và được so sánh ở mỗi lần build.

`kb build` chỉ chuẩn hoá phần trình bày — khoảng trắng không ngắt dòng, tab, khoảng trắng thừa cuối dòng, ký hiệu căn cột, dòng phân cách dư — rồi so sánh từng byte. Phép so sánh chạy theo cả hai chiều, nên một bảng bị sửa, bị mất, bị nhân đôi, bị đảo thứ tự hay bị bịa ra đều làm build thất bại.

Hệ quả thực tế: phần nội dung dễ sai nhất trong một tài liệu kỹ thuật lại chính là phần mà model về mặt cấu trúc không có khả năng làm hỏng.

2.4 Tài nguyên ảnh

Ảnh được trích trong lúc ingest và lưu theo địa chỉ nội dung tại `.kb/<doc>/assets/<sha256>.png` hoặc `.webp`. Cùng một chuỗi byte luôn cho cùng một tên file, nên ingest lại không bao giờ nhân bản tài nguyên, và một hình dùng lại ở nhiều chương chỉ được lưu một lần.

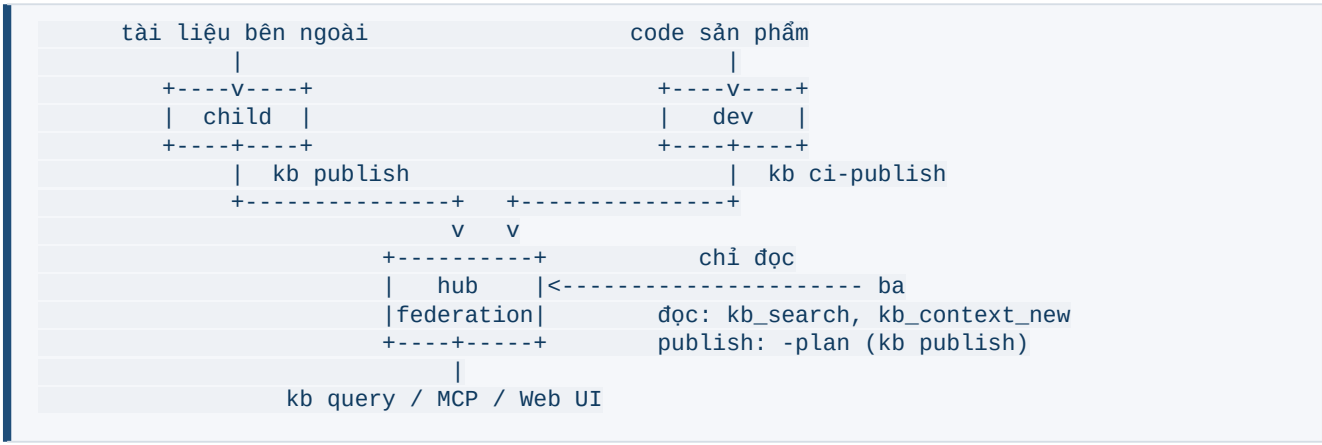
Mô tả ảnh không bao giờ được sinh ra. Mô tả lấy từ chính tài liệu gốc — ưu tiên caption của ảnh, nếu không có thì OCR phần chữ nằm trong ảnh. L3 mang tham chiếu ảnh Markdown tương đối; L2 mang đúng mô tả đó dưới dạng dòng `Figure: …` để ảnh cũng tham gia vào tìm kiếm từ khoá và tìm kiếm ngữ nghĩa.

---

3. Các loại repository

Strata không phải một repository. Nó là một tập nhỏ các vai trò repository, hợp lại thành một chuỗi cung ứng tri thức.

Loại	Tạo ra	Tiêu thụ	Publish lên hub
`hub`	`federation/`, dịch vụ tìm kiếm	publish từ mọi child	chính nó, và tuỳ chọn lên một hub cấp trên
`child`	tài liệu nghiệp vụ từ nguồn bên ngoài	—	có
`ba`	ticket và mission plan	hub, chỉ đọc	có (chỉ `-plan`)
`dev`	code sản phẩm, kèm tri thức về chính code đó	hub, chỉ đọc	có (`-code`, `-svc`)
`tester`	test case, script và báo cáo kiểm thử theo từng story	hub, chỉ đọc	có (chỉ `-test`)



`kb init --kind <kind>` dựng repository theo vai trò và ghi lựa chọn vào `.kb/config.yaml`. Kind không phải thứ trang trí: nó quyết định lệnh nào khả dụng, slash command và CI workflow nào được sinh ra, và MCP client được nối dây kiểu gì (stdio trên hub, HTTP ở những nơi còn lại).

Repo `tester` kiểm thử từng story trên một môi trường test dùng chung. Nó đọc ticket và cơ sở tri thức từ hub, viết test case và script Playwright, rồi publish đúng một tài liệu sinh tự động, `<repo_id>-test`: các test case và báo cáo `PASS` hoặc `FAIL` của từng vòng. Giống `ba`, nó không ingest tài liệu và không sửa tri thức. Nó chỉ đọc hub và chỉ publish tài liệu của chính mình, qua pull request. Giai đoạn 1 mới có phần việc của Tester; khâu bàn giao với Dev và BA, cùng `kb auto` cho `tester`, chưa được xây dựng.

3.1 Vì sao tách vai trò

Repo `child` ingest tài liệu từ **bên ngoài** repository. Repo `dev` **không ingest gì cả** và chỉ soạn tri thức về chính source code của nó. Tách như vậy để một repo sản phẩm có thể publish tri thức liên tục từ CI mà không bao giờ có được khả năng thêm tài liệu tuỳ ý vào kho chung.

Repo `ba` chỉ đọc đối với cơ sở *tri thức*. Nó trích dẫn, không bao giờ đóng góp, và không bao giờ ingest hay tóm tắt. Nhờ vậy yêu cầu nghiệp vụ truy vết được tới một phiên bản tri thức mà không trao cho quy trình làm yêu cầu quyền thay đổi tri thức đó. Thứ duy nhất nó publish là plan của chính nó, một tài liệu do máy sinh (`<repo_id>-plan`) chỉ liệt kê các mission và ticket mà BA đã đánh dấu sẵn sàng (mục 8). Plan nằm cạnh tri thức; nó không sửa tri thức.

---

4. Các thành phần

4.1 CLI `kb`

Một file thực thi duy nhất, nhóm theo giai đoạn vòng đời.

Nhóm	Lệnh
Dựng khung	`init`, `docker-setup`, `mcp-setup`
Biên soạn	`ingest`, `l3`, `summarize`, `status`, `build`, `approve`, `authored`
Truy hồi	`query`, `get`, `stats`, `tags`
Liên kết	`publish`, `ci-publish`, `reindex`, `doctor`, `assets`, `code-sync`
Trích dẫn	`context new`, `resolve`, `diff`
Tri thức code	`code-ingest`, `svc note`
Tri thức plan	`plan-ingest`, `mission next`, `ticket fetch`
Cổng kiểm tra	`ticket lint`, `ticket check`, `ticket stale`, `mission lint`, `pr lint`, `plan waves`
Dọn dẹp và xuất	`ticket tidy`, `ticket export`, `impl tidy`
Agent	`claim`, `pr` (`find`, `view`, `open`, `checks`, `merge`, `edit`; `pr lint` là một cổng ở trên), `auto`
Tài khoản hub	`user`
Đo lường	`usage`

CLI là bản hiện thực tham chiếu của mọi quy tắc trong tài liệu này. MCP server và Web UI là các mặt tiền khác trên cùng một thư viện, không phải bản cài đặt lại.

4.2 Pipeline ingest

```
PDF ──▶ parser ──▶ sectioner ──▶ scaffolder ──▶ summarizer ──▶ builder
       (docling)  (tiêu đề)     (L3+L1+L2)     (LLM CLI)     (cổng)
                                    │
                                    ├── ảnh ──▶ tài nguyên theo địa chỉ nội dung
                                    └── bảng ──▶ nguyên văn, vào cả L2 lẫn L3
```

Parse dùng docling để khôi phục bố cục, bảng và ảnh từ PDF.

Sectioning ưu tiên chính bookmark/outline của tài liệu; với `--no-bookmarks` thì chuyển sang tách theo mẫu tiêu đề, gom phần đầu sách vào `front-matter`, và sinh id dạng slug dễ đọc cho những tiêu đề không phân tích được. Mọi quyết định của bước tách đều được báo cáo — tiêu đề bị hạ cấp, id dự phòng, đánh số sai thứ tự, id trùng, bookmark bị bỏ sót — nên việc tách là kiểm toán được chứ không phải phép màu.

Scaffolding ghi L3 ngay lập tức và đầy đủ, tạo L2 với bảng đã nằm sẵn còn phần văn xuôi để trống, rồi ghi L1 với mọi section ở trạng thái `pending`. Tại thời điểm này tài liệu đã dùng được hoàn toàn ở mức L3 và hoàn toàn trung thực về phần còn thiếu.

Summarization là bước duy nhất có model tham gia. Nó gọi một LLM CLI chạy ngầm thay vì gọi API, nên chạy được trên gói thuê bao sẵn có. Nó bị ràng buộc bởi các quy tắc cố định: viết bằng chính ngôn ngữ của tài liệu gốc, không diễn đạt lại định danh hay con số, không đụng vào bảng, và thà giữ nguyên câu chữ gốc còn hơn bịa.

Build là cổng. Nó từ chối cho một kho chưa hoàn tất hoặc đã trôi dạt đi tiếp.

4.3 Chỉ mục tìm kiếm

`kb query` và MCP tool `kb_search` chạy truy hồi lai:

1. **Lọc trước theo tag ở L0.** Gần như miễn phí, và chính nó làm cho một federation lớn còn tìm được. Id tài liệu được đánh chỉ mục như một tag tổng hợp, nên người gọi thu hẹp về một tài liệu mà không cần cờ riêng.
2. **Nhánh từ khoá** — SQLite FTS5 trên văn bản section.
3. **Nhánh ngữ nghĩa** — KNN trên embedding, tuỳ chọn, có khi cài extra `embed`.
4. **Hợp nhất** — reciprocal rank fusion trên hai nhánh. Mỗi nhánh bị chặn ở 50 kết quả trước khi hợp nhất, và người gọi được báo khi có kết quả bị loại.
5. **Khống chế ngân sách** — nạp nội dung L2 của các kết quả đầu cho tới khi chạm ngân sách token. Section không bao giờ bị cắt giữa chừng, và chính điều đó giữ cho bảng trả về nguyên văn; vì vậy ngân sách chỉ mang tính khuyến nghị và kết quả đầu tiên luôn được trả về trọn vẹn.

Không có lời gọi model nào trong lúc tra cứu. Truy hồi là code thuần: nhanh, miễn phí và tất định.

Chỉ mục được lưu lại nhưng là dẫn xuất. Xoá đi và dựng lại từ Markdown bất cứ lúc nào cũng được — đó là lý do Markdown, chứ không phải chỉ mục, là nguồn chân lý.

4.4 Tiến trình dịch vụ

Một tiến trình phục vụ ba nhóm đối tượng qua HTTP:

Bề mặt	Đối tượng	Xác thực
`/mcp`	AI agent	Bearer token
`/api/…`	người gọi lập trình	Bearer token hoặc cookie phiên
`/ui`	con người	đăng nhập bằng tài khoản hub, lưu cookie phiên; token chỉ mở UI ở chế độ chỉ đọc

UI có menu **Ticket** nằm cạnh Document (`/ui/tickets`) cho các mission và ticket mà repo `ba` publish (mục 8.4).

Cũng tiến trình đó chạy qua stdio cho agent cục bộ. Ở mọi chế độ, hub là **bắt buộc** — lấy từ `--hub`, biến môi trường, hoặc `.kb/config.yaml`, theo đúng thứ tự đó — và server từ chối khởi động nếu không có. Nó chỉ tìm trong `federation/` của hub, không bao giờ tìm trong `.kb/` làm việc của chính nó. Chính quy tắc đó làm cho từ "đã publish" có ý nghĩa.

Mười bốn MCP tool được phơi ra: `kb_search`, `kb_get_section`, `kb_context_new`, `kb_resolve`, `kb_ticket_lint`, `kb_ticket_check`, `kb_ticket_stale`, `kb_ticket_fetch`, `kb_question_ask`, `kb_question_list`, `kb_question_answer`, `kb_mission_lint`, `kb_mission_next` và `kb_doctor_hub`. Ba tool `kb_question_*` làm việc với kho hỏi đáp ticket, và `kb_question_ask`, `kb_question_answer` là hai MCP tool duy nhất có ghi (xem 8.5); mười hai tool còn lại chỉ đọc.

---

5. Federation

5.1 Hub

Hub là một repository Git bình thường, có cùng bố cục `.kb/` như mọi repo khác, cộng thêm thư mục `federation/` do máy sinh:

```
hub/
├── .kb/                          tài liệu của chính hub, nếu có
└── federation/
    ├── index.yaml                L0 tổng hợp trên mọi entry
    ├── registry.yaml             tuỳ chọn: repo nào được publish id nào
    ├── <repo-a>/                 bản sao đầy đủ .kb/ của repo A (L0-L3)
    ├── <repo-b>/
    └── <mid-hub>/<repo-c>/       entry lồng nhau từ một chuỗi hub cấp trên
```

Mỗi repository tham gia đều sao chiếu **toàn bộ** kho của nó — cả bốn lớp, không phải ảnh chụp rút gọn — nên hub trả lời được mọi truy vấn mà không cần với ngược về repo nguồn. Giữa hub và các child không có ràng buộc lúc chạy.

5.2 Publish

`kb publish` sao chiếu `.kb/` vào `federation/<repo-id>/` và dựng lại chỉ mục tổng hợp. Đích đến lấy từ cấu hình chứ không từ cờ dòng lệnh, nên một repository không thể vô tình publish nhầm hub.

Chế độ	Khi nào áp dụng
Pull request (`--pr`)	Đường đi thông thường. Cần git remote và `gh` mở được PR trên host đó.
Commit trực tiếp (`--direct`)	Bị từ chối trên hub *được quản trị* có remote. Dành cho hub thuần cục bộ.
Tự động	Chỉ chọn direct với hub không có remote; còn lại là PR, hoặc từ chối rõ ràng.

Publish chỉ sao chép **artefact** — tài liệu, chỉ mục, manifest, tài nguyên ảnh. File cấu hình và dotfile bị giữ lại và được nêu tên trong cảnh báo. Đây là một thuộc tính an toàn chứ không phải sự gọn gàng: `.kb/config.yaml` chứa các trường `hub:` và `intake:`, và một token lỡ nằm trong file không bao giờ được sao chiếu lên hub.

`kb ci-publish` là đường đi cho CI. Nó chạy bên trong chính GitHub Actions job của repo publish, xác thực với dịch vụ intake bằng OIDC của GitHub Actions, nhờ vậy không cần secret dài hạn ở bất cứ đâu. Repo `child`, `dev` và `ba` đều publish qua đường này; luồng của `ba` được mô tả ở mục 8.2.

5.3 Federation nhiều tầng

Một hub có thể tự khai báo hub cấp trên. Publish một repo như vậy sẽ sao chiếu `federation/` của nó — chứ không phải `.kb/` của nó — lên hub trên, giữ nguyên bố cục lồng nhau. Độ sâu không giới hạn, id entry trở thành đường dẫn, và một chuỗi có thể khiến nội dung quay vòng sẽ bị từ chối với lỗi phát hiện chu trình rõ ràng.

Điều này cho tìm kiếm một cách khống chế phạm vi rất tự nhiên: truy vấn hub của nhóm thì ra tri thức của nhóm, truy vấn hub gốc thì ra tất cả. Không cần logic lọc nào, vì chính topology đã diễn đạt phạm vi.

5.4 Toàn vẹn trên hub

- `federation/index.yaml` ghi hash nội dung cho từng snapshot, nên `kb doctor` phát hiện được nội dung bị sửa trực tiếp trên hub thay vì qua publish.
- `kb reindex` dựng lại chỉ mục tổng hợp từ các entry khi nó lệch.
- `kb doctor` báo lỗi cấu trúc, sai lệch giữa manifest và file, file mà một lượt publish không bao giờ ghi ra, bản clone hub còn lưu credential trong git config của chính nó, và git cũ hơn mốc 2.31.

5.5 Đẩy tài nguyên ảnh sang S3

Hub có thể khai báo asset store dùng S3. Khi đó publish sẽ tải ảnh lên bucket thay vì commit vào repo, đồng thời ghi lại những gì đã chuyển hướng bên cạnh mỗi tài liệu. Dịch vụ phục vụ ảnh đã chuyển hướng thẳng từ bucket, kèm cache đĩa cục bộ. Mặc định vẫn giữ ảnh trong Git như mọi file khác; lựa chọn này theo từng hub và trong suốt với người gọi.

---

6. Trích dẫn ghim phiên bản

Tri thức thay đổi. Một yêu cầu viết dựa trên bản sửa đổi quý trước không được phép âm thầm mang nghĩa khác đi.

Khối `kb-context` là một trích dẫn máy đọc được, nhúng vào ticket, spec hoặc tiêu chí chấp nhận. Nó ghi lại commit của hub tại thời điểm viết và một tập tham chiếu section có định danh repo.



- **`ok`** — nội dung đã ghim vẫn khớp với nội dung đang publish.
- **`stale`** — kho đã được sửa đổi sau khi trích dẫn được lấy. `kb diff` cho biết chính xác những gì đã đổi giữa hai bản sửa đổi: tiêu đề, tóm tắt L1, lát cắt L2, bản gốc L3, section được thêm và bị xoá, và cả việc manifest có bị

đảo thứ tự hay không.

- **`broken`** — tham chiếu không còn phân giải được nữa.

`kb doctor --context` chạy đúng phép kiểm tra đó trong CI, nên một yêu cầu đã cũ được cảnh báo trước khi merge thay vì bị phát hiện lúc đang lập trình.

---

7. Tri thức về code

Một repo `dev` publish hai tài liệu về chính nó. Hai tài liệu này tách nhau có chủ đích.

Tài liệu	Nguồn gốc	Sinh lại	Cần review	Tag
`<repo_id>-code`	`kb code-ingest`, tất định, không model	sinh lại khi cần từ working tree; CI publish ở mỗi tag `kb publish`	không	`code, generated`
`<repo_id>-svc`	model soạn từ bằng chứng đó, người sửa lại	không bao giờ	có	`code, curated`

Chúng không thể dùng chung một tài liệu vì ba lý do độc lập: `-code` bị ghi đè toàn bộ trong khi `-svc` tích luỹ dần; `-svc` mang section `pending` trong lúc seed còn `-code` thì luôn phải build sạch; và `-code` có thể được công tắc auto-merge của repo tự động merge còn `-svc` thì cần người review (làm tại máy, trước khi publish).

`-code` không bao giờ được commit: revision của nó nằm trong manifest, không nằm trong `index.yaml`, nên index đã commit của repo dev không đổi khi code đổi.

Chúng nối với nhau bằng **id section**: `svc.<name>` tồn tại ở cả hai. `-code` cung cấp alias, nhãn và công nghệ của một service; `-svc` cung cấp trách nhiệm của service đó. Ghép lại chúng điền đủ bốn tham số của `Container(...)` trong C4, nhờ đó một BA agent vẽ được sơ đồ kiến trúc chính xác từ tri thức đã publish.

7.1 Trích xuất

`kb code-ingest` được tổ chức theo **loại artefact chứ không theo ngôn ngữ lập trình**, vì Strata được áp dụng trên nhiều dòng dự án và một bộ extractor chỉ phục vụ một stack sẽ khiến phần lớn repository không trích được gì.

Extractor	Sinh ra	Đọc
`services`	`svc.<name>`	file compose, Dockerfile, manifest k8s, file solution và workspace
`deps`	`dep.<ecosystem>`	manifest của Python, JavaScript, Java, .NET, Go, PHP, Rust, Swift và Dart
`commands`	`cmd.<purpose>`	script trong package, Makefile, tox, shell script, bước run trong CI
`tree`	`struct.tree`	các file Git theo dõi, nên thư mục bị ignore không bao giờ xuất hiện
`schema`	`db.<table>`	migration SQL, Prisma, Alembic, EF, và file SQLite được nếu tên tường minh
`integrations`	`int.<name>`	**chỉ tên khoá** biến môi trường
`api`	`api.<tag>`	hợp đồng OpenAPI và Swagger nằm trong repo
`routes`	`route.<slug>`	route HTTP khai báo trong mã nguồn (cần extra `code`)
`pages`	`page.<slug>`	route trang UI; router theo hệ thống file chạy được không cần extra `code`
`jobs`	`job.<slug>`	job khai báo bởi scheduler của framework (cần extra `code`)
`symbols`	`sym.<slug>`	tên được export, member của enum và object, CSS custom property (cần extra `code`)

`tree` còn ghi `struct.files.<dir>`, và mọi extractor báo vào một section `struct.coverage` cho biết từng khu vực được quét `complete`, `partial` hay `not scanned`; `jobs` không bao giờ là `complete`. Bốn extractor đọc mã nguồn dùng tree-sitter để phân tích cú pháp, từ extra tuỳ chọn `strata-kb[code]`.

Đầu ra là một tài liệu bốn lớp bình thường, nên build, query, federation và Web UI đọc được mà không cần xử lý đặc biệt.

7.2 Hai thuộc tính an toàn

Đây là thuộc tính của thiết kế, không phải tiện ích có thể nới lỏng.

1. **Chỉ lấy khoá, không bao giờ lấy giá trị.** File môi trường duy nhất mà extractor integrations mở là file example/sample/template — không bao giờ là `.env` thật — và biểu thức đọc file đó không có nhóm bắt nào quanh phần giá trị, nên không có gì để rò rỉ dù dòng khớp chứa gì đi nữa. Khối `environment:` trong compose **có** được đọc và có chứa giá trị thật, nhưng chỉ tên khoá được giữ lại.
2. **Đầu vào database phải tường minh.** Schema SQLite chỉ được đọc từ đường dẫn nêu trên dòng lệnh. Một file database nháp hay fixture nằm trong repo không bao giờ vô tình trở thành tri thức đã publish của công ty.

7.3 Tính tất định

Extractor là hàm thuần trên cây file Git theo dõi cộng với đầu vào được nêu tên. Section, dependency và bảng đều sắp xếp tất định, thứ tự cột được giữ nguyên vì nó mang ý nghĩa, khoá khi ghi ra được sắp xếp, dấu phân cách đường dẫn và ký tự xuống dòng được chuẩn hoá. Chạy lại trên cùng một commit, cùng một nền tảng sẽ cho file giống nhau từng byte — chính điều đó làm cho lượt publish trong CI thực sự là no-op khi cây file không đổi.

Hai giới hạn được nói thẳng: một vài phép khớp glob chuẩn hoá chữ hoa/thường theo hệ điều hành, nên đảm bảo này là theo từng nền tảng chứ không phổ quát (CI luôn chạy Linux nên thứ được publish vẫn ổn định); và `revision` trong manifest suy ra từ commit HEAD trong khi cây làm việc có thể đang có thay đổi chưa commit — trường hợp này được phát hiện và báo bằng cảnh báo `dirty_tree`.

---

8. Tri thức về plan

Repo `ba` là bên publish thứ ba. Thứ nó publish không phải tri thức nghiệp vụ và cũng không phải tri thức về code mà là **plan**: các mission và ticket mà BA đã review, dưới dạng một tài liệu do máy sinh, `<repo_id>-plan`.

Tài liệu	Nguồn gốc	Sinh lại	Cần review	Gắn tag
`<repo_id>-plan`	`kb plan-ingest`, tất định, không model, chạy offline	theo yêu cầu từ `missions/` và `tickets/`	không: Definition of Ready được tick đủ chính là review	`plan, generated`

Về mặt cơ học nó theo `-code` ở mọi điểm. Nó không bao giờ được commit (`.kb/*-plan/` bị gitignore, `.kb/index.yaml` do máy sinh của repo `ba` cũng vậy), revision của nó nằm trong manifest, và nó là một tài liệu bốn lớp bình thường nên build, query, federation và Web UI đọc được mà không cần xử lý đặc biệt. Các section của nó được ghi là `summarized`, không bao giờ `pending`, nên `kb build` pass mà không cần model.

8.1 Cái gì đủ điều kiện, và hình dạng tài liệu

Một mission được publish khi qua `kb mission lint` về cấu trúc **và** mọi ô trong `## Definition of Ready` đã được tick. Một ticket cần điều tương tự từ `kb ticket lint`, và `kb mission next` không được hạ nó xuống `draft` vì mission cha đã đi tiếp. Việc phân giải trích dẫn **không** phải là cổng: nó cần hub, và lint ở pull request đã cưỡng chế việc đó. Mỗi mục bị bỏ ra ngoài được báo là `skipped: <file> — <reason>` và không bao giờ làm lệnh fail, nên một mục chưa sẵn sàng đơn giản là không được publish, và mục thôi đủ điều kiện sẽ rời đi ở lần publish kế tiếp.

Tài liệu có một nhóm cho mỗi mission và một nhóm `tickets` cho phần còn lại. Section là `## <id> <title>`, với `M-<slug>-US<n>` là id của một story. L3 là nguồn đã bỏ tiêu đề, hạ heading một cấp và bỏ hai mục nội bộ của BA `## Review record` và `## Definition of Ready`; L2 giữ nguyên từng byte mọi bảng (nên bất biến về bảng vẫn đúng mà không phải nới `kb build`) cộng thêm phần văn xuôi tóm tắt mục đó. Một dòng bắt đầu bằng `##` nằm trong khối code sẽ cắt section với mọi người đọc, nên mục như vậy bị bỏ qua cho tới khi dòng đó được thụt vào.

8.2 Luồng publish

Publish là theo yêu cầu. Intake của hub chỉ nhận tag `kb-publish/*` (không bao giờ nhận push nhánh), nên plan đi theo cùng con đường dựa trên tag mà repo `child` dùng:

```
 BA chạy kb publish
      |  kb plan-ingest + kb build      (bản xem trước cục bộ = thứ CI sẽ gửi)
      |  gắn tag kb-publish/<ts>, đẩy tag
      v
kb-publish.yml (kích hoạt bởi tag, OIDC)
      |  kb plan-ingest + kb build + kb ci-publish
      v
intake: xác minh token OIDC, tra repo trong
        federation/registry.yaml  -->  PR trên hub
      |  review + merge
      v
federation/<repo_id>/<repo_id>-plan  -->  kb query / MCP / Web UI
```

Mục trong registry, không bao giờ là payload, quyết định repo id mà nó được ghi. (Ghim `workflow:` tuỳ chọn của nó được so với toàn bộ `job_workflow_ref`, trong đó có tag, nên để trống với publish kích hoạt bằng tag.) Một lần merge vào `main` (như đã scaffold) có đụng tới `missions/` hoặc `tickets/` sẽ gắn tag cho commit đã merge rồi publish (`kb-publish-request.yml`); BA cũng có thể chạy `/kb-publish`, lệnh này cho xem thay đổi trước khi hỏi. Repo chưa cấu hình `intake:` sẽ quay về đường snapshot.

8.3 Drift: giữ mission và ticket luôn cập nhật

Một mission được viết một lần rồi thực tế dịch chuyển. Mỗi mission và ticket mang `> Revision: <N>` cùng một `## Change log` chỉ thêm vào, và hai phép kiểm tra phát hiện drift. Cả hai chạy trong workflow `kb-doctor` hằng ngày qua `kb ticket stale`, lệnh này giữ một issue `kb-stale-citations`:

```
 Drift KB-context       section hub được trích, ở commit hub đã ghim
                        so với worktree của hub    -->  ok | stale | broken

Drift architecture     các id được trích trong Services & order / Technical
                       grounding (svc.* db.* api.* int.* cmd.*, trừ [NEW: D<n>])
                            |
      Grounded on: <repo>-code @ <rev>   (commit của repo dev)
                            |  duyệt lịch sử hub của manifest -code;
                            |  commit hub mới nhất có revision == <rev>
                            v
      <repo>-code §<id> @ <commit hub đó>  --> cùng bước resolve
                            -->  stale | broken, kind "architecture"
      không commit hub nào mang <rev>  -->  một phát hiện, "baseline not on hub"
```

Drift được xét **theo từng section được trích**, không bao giờ theo revision của cả tài liệu: revision của `-code` đổi ở mỗi commit dev, nên so sánh nó sẽ gắn cờ mọi mission ở mỗi lần merge. Một lần merge không đụng tới thứ mission

trích dẫn thì không tạo phát hiện nào.

`kb ticket check` theo cùng quy tắc đó. Revision `Grounded on:` bị lệch không bao giờ tự nó là lỗi: check tìm baseline theo cùng cách, rồi để các bước kiểm tra fact của ticket trên `-code` hiện tại quyết định. Mọi fact được trích vẫn đúng → PASS, kèm `[note]` khi các section được trích không đổi, hoặc `[warn]` kèm diff cho mỗi section đã đổi. Một fact được trích đã mất → FAIL với `re-run /sa-ticket-ground`. Lỗi về quyết định không bao giờ mang dòng đó.

Một lần sửa đổi (`/ba-mission-plan --amend`, `/ba-ticket-author --amend`) ghi lại trigger của nó (`kb-context`, `architecture`, `scope`, `delivery`), tăng revision và thêm một dòng change log. Một lần amend ticket luôn chạy `kb ticket check` và gọi SA với trigger `architecture`, hoặc khi check fail với `re-run /sa-ticket-ground`; một lần amend mission chỉ gọi SA và check với `architecture`. Story đã giao không bao giờ bị viết lại: thay đổi đến dưới dạng story mới hoặc quyết định thay thế (`SUPERSEDED`), và một ticket ghim `> Parent mission: M-<slug> @ rev <N>` bị `kb mission next` báo `draft` khi mission của nó đã đổi story đó.

8.4 Trạng thái trên hub

Trạng thái story được suy ra, chỉ đọc, từ những gì hub đã có: `done` khi id nằm trong một dòng `hist.*` của `-svc` của repo dev, `ready` khi ticket đã publish, `not ready` khi chưa, `blocked` kèm lý do. `kb mission next --from-hub` tính nó ở CLI; menu **Ticket** của Web UI hiển thị nó, và tài liệu `*-plan` nằm ở menu Ticket, không nằm ở Document, và trang tổng quan đếm theo từng loại. Không có gì được lưu và không có đường ghi. Theo chiều ngược lại, một dev chỉ có id ticket chạy `kb ticket fetch`, lệnh này đọc ticket từ `-plan` của hub và khôi phục dạng nguồn của nó.

Khi Dev làm xong một story, báo cáo kiểm thử và claim của Tester bổ sung thêm bốn trạng thái, cũng được suy ra và không bao giờ được lưu: `testing` khi một claim của Tester trên story đang được giữ, `closed` khi báo cáo mới nhất đã publish là `PASS` (trạng thái cuối: về sau không gì đổi được nữa), `reopened` khi là `FAIL` (các ticket con bị nêu là có lỗi cũng hiện như vậy), và `fix-done` khi lịch sử Dev của một ticket con đã mở lại ghi nhận bản sửa của vòng đó. Chỉ các vòng `PASS` và `FAIL` được publish.

8.5 Câu hỏi về ticket

Dev gặp câu hỏi nghiệp vụ khi thiết kế hoặc lập kế hoạch cho một ticket đã publish sẽ hỏi BA qua `kb_question_ask`; câu hỏi được lưu trong SQLite tại `<hub root>/.kb-qa/qa.db`, cạnh `.kb-auth/` và nằm ngoài git. Đây là thứ duy nhất hub lưu mà không nằm trong git, nên ba tool hỏi đáp chỉ chạy với chính hub, tức server HTTP của hub dùng chung hoặc server stdio chạy trong checkout của hub (`--kb <hub>/.kb`); mọi đích khác đều bị từ chối: server stdio của repo Dev, hay URL hub đọc qua bản clone cache cục bộ, trả lời `Q&A needs the shared hub server — point MCP at it (kb mcp-setup)`, vì kho ở đó không phải kho dùng chung.

Trạng thái câu hỏi được suy ra, không lưu. Câu hỏi *open* cho đến khi có câu trả lời. Câu trả lời làm thay đổi ticket ghi lại revision mang nó (`answer_rev`) và các section đã đổi; câu hỏi ở trạng thái *chờ publish* cho đến khi bản ticket trên hub (dòng `> Revision: N` của chính nó, không phải git HEAD của hub) đạt revision đó, rồi mới là *answered*. Menu **Q&A** của Web UI hiển thị chúng ở chế độ chỉ đọc. BA trả lời bằng `/ba-answer-questions`: skill sửa ticket (Trigger `question` trong change log), để BA tick lại Definition of Ready, ghi câu trả lời và không bao giờ commit, push hay publish; phía Dev nêu câu hỏi nghiệp vụ từ `dev-design` và `dev-plan`, còn câu hỏi kỹ thuật chuyển cho `dev-sa`.

---

9. Mô hình tin cậy

Lập luận về tính đúng đắn của Strata là một chuỗi bốn mắt xích, mỗi mắt xích đều có thể hỏng một cách ồn ào, và không mắt nào được phép bỏ qua.



Hãy để ý điều **không** được khẳng định. Model không được tin cậy, và thiết kế cũng không cần nó đáng tin: nó bị giới hạn trong văn xuôi, đầu ra của nó bị chặn bởi các phép kiểm tra tự động, và nó không publish được. Tương tự, registry trên hub là **hàng rào chống nhầm lẫn**, không phải cơ chế xác thực — URL remote của bên publish là do chính bên đó tự khai. Branch protection trên hub và intake OIDC mới là những biện pháp thực sự có hiệu lực. Điều này cũng đúng với plan của repo `ba`: thứ được gửi đi do Definition of Ready mà một con người tick chọn ra, trên một tag từ `kb publish` của BA hoặc từ workflow chạy khi merge, và nó vẫn chỉ tới tay người đọc qua một pull request trên hub.

9.1 An toàn của dịch vụ

- Mọi phản hồi đều mang content security policy, chặn nhúng frame, nosniff, chính sách referrer và permissions; HSTS trên https.
- Cookie phiên của UI là giá trị đã ký, có hạn, dùng khoá `<hub>/.kb-auth/session.key` — không bao giờ là bearer token, thứ mà mọi repo con đều giữ. Cookie mang tên người dùng chứ không mang vai trò; vai trò được đọc từ CSDL tài khoản ở mỗi request. Tài khoản nằm trong `.kb-auth/`, không bao giờ được commit.
- Trang Registry (`/ui/registry`) chỉ dành cho admin, nên bearer token xem được UI nhưng không bao giờ sửa được registry. Mỗi lần sửa là một commit trên nhánh mặc định của hub, đẩy bằng GitHub App của intake, không bao giờ force-push. Công tắc `auto_merge` của repo khiến intake merge cả PR publish ngay sau khi mở; merge thất bại thì PR vẫn mở và publish vẫn `done` với `merged: false`.
- Kho hỏi đáp ticket (`<hub>/.kb-qa/qa.db`, không bao giờ được commit) chỉ được ghi bởi ba MCP tool `kb_question_*`, và chỉ trên chính hub (không phải bản clone cache). Mọi bên gọi mà hub xác thực đều được ghi; tên họ khai là tự khai, nên chỉ để theo dõi ai nói gì và không xác thực ai. Mỗi ticket có tối đa 50 câu hỏi mở cùng lúc.
- Một lần thử bearer token thất bại dùng chung bucket giới hạn tần suất với form đăng nhập, nên không thể né khoá bằng cách chuyển sang header.
- Khi đứng sau reverse proxy, **bắt buộc** phải cấu hình số proxy tin cậy. Mặc định là bỏ qua header chuyển tiếp và khoá theo địa chỉ socket; để nguyên mặc định khi đứng sau proxy sẽ khiến mọi người gọi dùng chung địa chỉ của proxy, và một người gọi có thể làm cạn giới hạn của tất cả.
- Xác thực HTTP là bearer token cho `/api`, `/mcp` và CI cùng tài khoản hub cục bộ cho UI — không OAuth, không SSO. Mức đó đủ cho mạng nội bộ và chưa sẵn sàng cho Internet công cộng.

---

10. Các mô hình triển khai

10.1 Một nhóm

```
người soạn ──▶ repo child ──▶ repo hub ──▶ một tiến trình dịch vụ
                                              (MCP + REST + UI)
```

Một hub, một hoặc vài child, một dịch vụ. Hub có thể chính là repository soạn nội dung. Đây là cấu hình khởi đầu và nó mở rộng xa hơn phần lớn nhóm nghĩ, vì dịch vụ không giữ trạng thái còn kho chỉ là một bản clone Git.

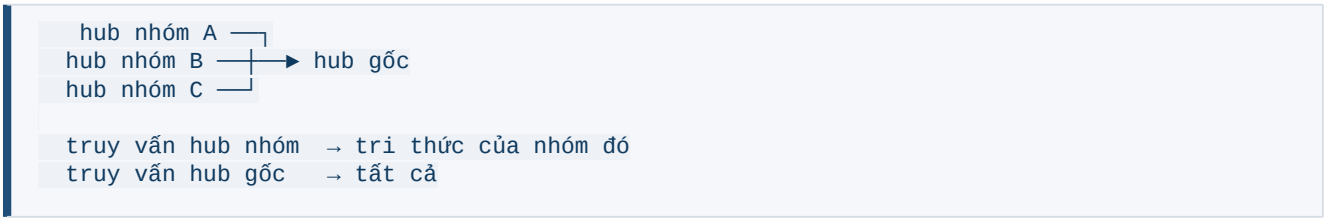
10.2 Toàn tổ chức

```
    nhóm nghiệp vụ        nhóm sản phẩm         nhóm yêu cầu
   child × N             dev × N               ba × N
       \                    |                    /
```



Mọi repository publish về một hub; mọi bên tiêu thụ đọc từ đó. Repo `ba` và `dev` không đóng góp nội dung nghiệp vụ: repo `dev` chỉ publish tri thức về chính nó, còn repo `ba` chỉ publish plan của nó.

10.3 Tổ chức liên kết nhiều tầng



Hub trung gian cho mỗi đơn vị một phạm vi tìm kiếm riêng mà vẫn gộp lên một góc nhìn chung toàn tổ chức. Phạm vi được diễn đạt bằng topology chứ không bằng việc lọc lúc truy vấn.

---

11. Đặc tính vận hành

Thuộc tính	Giá trị
Trạng thái lúc chạy	Không có. Mọi artefact là file được quản lý phiên bản.
Chi phí truy vấn	Không gọi model. Đọc chỉ mục cục bộ cộng đọc file.
Chi phí ingest	Một lần cho mỗi tài liệu; lần chạy đầu tải mô hình bố cục (~500 MB) rồi cache lại.
Chi phí tóm tắt	Một lời gọi model cho mỗi section, một lần, chạy lại được khi cần.
Giới hạn mở rộng	Thực tế là kích thước một bản clone Git và một chỉ mục FTS5 — vài chục nghìn section là chuyện bình thường.
Cách hỏng	Suy giảm chứ không im lặng: kho hỏng thì build fail; trích dẫn cũ thì báo `stale`; nhánh ngữ nghĩa không dùng được thì cảnh báo và lui về tìm kiếm từ khoá.
Nền tảng	Python 3.11–3.13, Linux và Windows, kiểm thử ở mỗi bản phát hành.
Phục hồi	Xoá chỉ mục rồi dựng lại; clone lại hub; publish lại từ repo nguồn.

---

12. Điểm mở rộng

Strata được thiết kế để mở rộng theo bốn đường nối, mỗi đường hấp thụ thay đổi mà không phải động vào lõi.

1. **Chiến lược tách section.** Tách theo bookmark và tách theo mẫu tiêu đề là hai hiện thực của cùng một giao diện. Một họ tài liệu có cấu trúc riêng sẽ có hiện thực thứ ba, chứ không phải một nhánh đặc biệt trong pipeline.
2. **Extractor.** `kb code-ingest` điều phối tới các extractor độc lập theo loại artefact. Một hệ sinh thái mới là một extractor mới và một tiền tố section mới, không phải một thay đổi định dạng tài liệu.
3. **Nhánh truy hồi.** Nhánh từ khoá và nhánh ngữ nghĩa được hợp nhất một cách tổng quát. Tín hiệu xếp hạng thứ ba sẽ tham gia vào bước hợp nhất chứ không thay thế pipeline.
4. **Tiền tố id section.** Bảng tiền tố là hợp đồng công bố mà công cụ agent dựa vào. Loại tri thức mới nhận tiền tố mới; tiền tố cũ không bao giờ đổi nghĩa.

---

13. Tóm lại

Kiến trúc của Strata là một ý tưởng duy nhất được áp dụng nhất quán: **đặt phần việc tốn kém và dễ sai vào nơi có thể kiểm tra được.**

Tách section là việc máy móc, nên báo cáo và kiểm toán được. Bảng là việc máy móc, nên diff được. Văn xuôi do model sinh, nên bị ràng buộc bởi quy tắc và bởi kiểm tra chất lượng. Publish là việc của con người, nên nó là một pull request. Truy hồi là việc máy móc, nên nó miễn phí và lặp lại được.

Không chỗ nào trong hệ thống yêu cầu phải tin một ai — người hay model — ở nơi mà lẽ ra có thể chạy một phép kiểm tra.